

LAB EXERCISE – 7

UCS3513 System Design Laboratory – Autocomplete Search System using Trie and Redis

AIM

To design and implement an autocomplete search system using prefix matching. The system uses a Trie data structure for fast prefix search, returns the top-K suggestions by frequency, and caches results in Redis to improve response time for repeated queries.

PROBLEM STATEMENT

A search engine must suggest relevant terms as the user types. For example, when the user types app, the system should return apple, application, appointment, and app store. Searching the full collection every time is slow, so we use a Trie for efficient prefix lookup and Redis to cache frequent prefixes.

SYSTEM ARCHITECTURE

The application is a Spring Boot 3.2.5 service. It exposes a REST endpoint `/api/search`. When a request arrives, the service first checks Redis. If the prefix is cached, it returns immediately. If not, it walks the Trie, collects matching terms, sorts them by frequency, stores the top-K result in Redis, and returns it.

Fig 1. Autocomplete system architecture

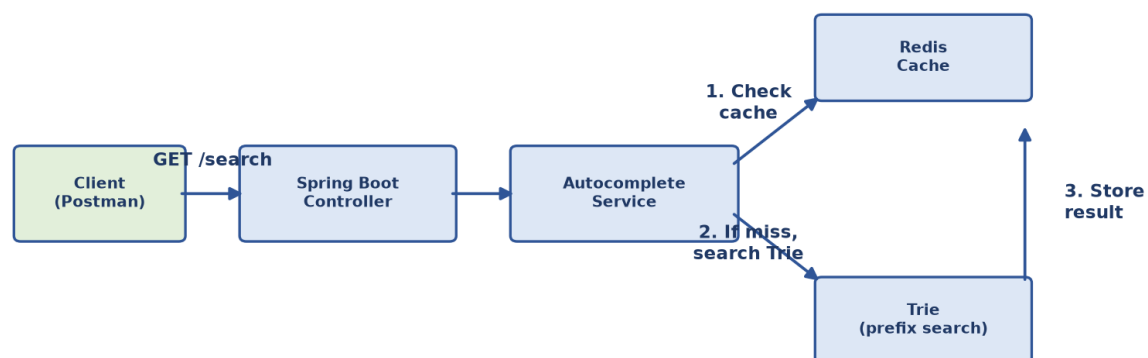


Fig 1. Autocomplete system architecture

ALGORITHM AND DATA STRUCTURE

The whole implementation fits inside one AutocompleteService class. It contains an inner Trie class, an inner TrieNode class, and a Suggestion class. The service seeds sample search terms, exposes a search method, and uses Spring Data Redis to cache results.

Fig 2. Cache hit / cache miss flow

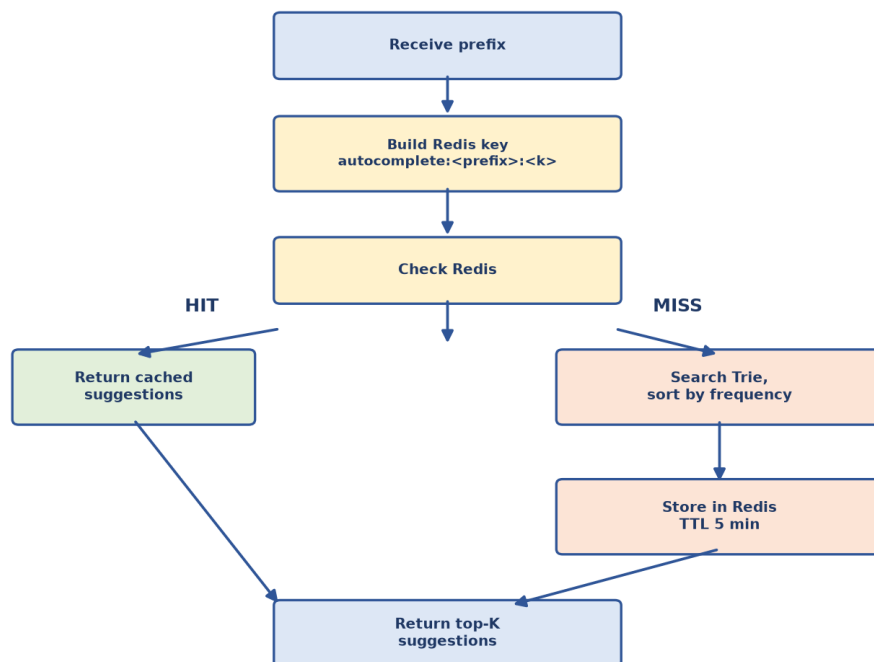


Fig 2. Cache hit / cache miss flow

Fig 3. Trie structure for prefix "app"

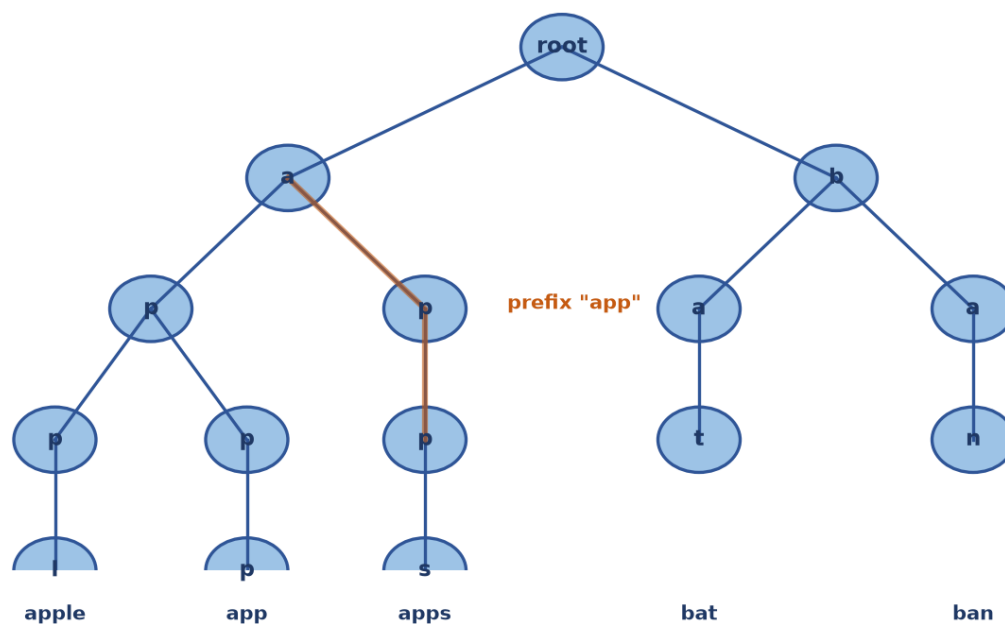


Fig 3. Trie structure for prefix app

- Trie (inner class): a tree where each edge is a character. A path from the root to a node represents a prefix; a node with a non-null word stores a search term and its frequency.
- Prefix search: walk the Trie following the prefix characters. From the prefix node, collect all descendant words using DFS.
- Top-K selection: sort the collected words by frequency descending and keep the first K.
- Redis cache: store the top-K suggestion list under a key auto:<prefix>:<k> with a TTL of 5 minutes (300 seconds).

- Cache hit: Redis returns the list directly. Cache miss: search the Trie and then write the result to Redis.

CODE

```
package com.ssn.autocomplete;

import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Service;

import java.util.*;
import java.util.concurrent.TimeUnit;

@Service
public class AutocompleteService {

    private final Trie trie = new Trie();
    private final StringRedisTemplate redis;

    public AutocompleteService(StringRedisTemplate redis) {
        this.redis = redis;
        // sample search terms with frequencies
        trie.insert("apple", 1200);
        trie.insert("application", 950);
        trie.insert("appointment", 800);
        trie.insert("app store", 600);
        trie.insert("app", 500);
        trie.insert("banana", 700);
        trie.insert("band", 450);
        trie.insert("bandwidth", 300);
        trie.insert("cat", 900);
        trie.insert("caterpillar", 350);
        trie.insert("category", 400);
    }

    /** Return top-K suggestions for a prefix. Redis cache first, Trie on miss. */
    public List<String> search(String prefix, int k) {
        String key = "auto:" + prefix.toLowerCase() + ":" + k;

        // 1. Check Redis
        List<String> cached = redis.opsForList().range(key, 0, -1);
        if (cached != null && !cached.isEmpty()) {
            return cached;
        }
    }
}
```

```
// 2. Search Trie
List<Suggestion> list = trie.findTopK(prefix, k);

// 3. Build result
List<String> result = new ArrayList<>();
for (Suggestion s : list) {
    result.add(s.term + " (freq=" + s.freq + ")");
}

// 4. Cache with 5-minute TTL
if (!result.isEmpty()) {
    redis.opsForList().rightPushAll(key, result);
    redis.expire(key, 300, TimeUnit.SECONDS);
}

return result;
}

/** Simple Trie with prefix search and top-K by frequency. */
static class Trie {
    TrieNode root = new TrieNode();

    void insert(String word, long freq) {
        TrieNode node = root;
        for (char c : word.toLowerCase().toCharArray()) {
            node = node.child.computeIfAbsent(c, x -> new TrieNode());
        }
        node.word = word;
        node.freq = freq;
    }

    List<Suggestion> findTopK(String prefix, int k) {
        TrieNode node = root;
        for (char c : prefix.toLowerCase().toCharArray()) {
            node = node.child.get(c);
            if (node == null) return new ArrayList<>();
        }

        List<Suggestion> list = new ArrayList<>();
        collect(node, list);

        list.sort((a, b) -> Long.compare(b.freq, a.freq));
        return list.size() > k ? list.subList(0, k) : list;
    }
}
```

```
void collect(TrieNode node, List<Suggestion> list) {
    if (node == null) return;
    if (node.word != null) list.add(new Suggestion(node.word, node.freq));
    for (TrieNode child : node.child.values()) {
        collect(child, list);
    }
}

static class TrieNode {
    Map<Character, TrieNode> child = new HashMap<>();
    String word;
    long freq;
}

static class Suggestion {
    String term;
    long freq;
    Suggestion(String term, long freq) {
        this.term = term;
        this.freq = freq;
    }
}
}
```

CONTROLLER CODE

```
package com.ssn.autocomplete;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api")
public class AutocompleteController {

    private final AutocompleteService service;

    public AutocompleteController(AutocompleteService service) {
        this.service = service;
    }
}
```

```
@GetMapping("/health")
public ResponseEntity<String> health() {
    return ResponseEntity.ok("Autocomplete service is up");
}

@GetMapping("/search")
public ResponseEntity<List<String>> search(
    @RequestParam String prefix,
    @RequestParam(value = "k", defaultValue = "5") int k) {
    return ResponseEntity.ok(service.search(prefix, k));
}
}
```

CONFIGURATION

```
spring.application.name=autocomplete
spring.data.redis.host=localhost
spring.data.redis.port=6379
server.port=8080
```

ENDPOINTS

Method	Path	Description
GET	/api/health	Health check
GET	/api/search?prefix=app&k=4	Return top-K autocomplete suggestions

TEST OUTPUT

The application was built and run on a Windows host using Maven 3.9.16 and Java 21, with Redis on localhost:6379. The following curl requests were executed:

```
Health check:
HTTP/1.1 200
Autocomplete service is up

Search: /api/search?prefix=app&k=4 (first request - Trie + Redis cache miss)
HTTP/1.1 200
["apple (freq=1200)","application (freq=950)","appointment (freq=800)","app store (freq=600)"]

Search: /api/search?prefix=app&k=4 (second request - Redis cache hit)
```

```
HTTP/1.1 200
["apple (freq=1200)","application (freq=950)","appointment (freq=800)","app store (freq=600)"]

Search: /api/search?prefix=ba&k=3
HTTP/1.1 200
["banana (freq=700)","band (freq=450)","bandwidth (freq=300)"]

Search: /api/search?prefix=ca&k=3
HTTP/1.1 200
["cat (freq=900)","category (freq=400)","caterpillar (freq=350)"]
```

ANALYSIS

- Role of Trie: the Trie stores all search terms and supports fast prefix lookup in $O(\text{length of prefix})$ to reach the prefix node, followed by DFS to collect suggestions.
- Role of Redis: caches top-K results for hot prefixes, avoiding repeated Trie traversal and sorting.
- Top-K suggestions: sorting by frequency ensures the most-searched terms appear first.
- Cache hit path: the service builds a Redis key, fetches the cached list, and returns it immediately.
- Cache miss path: the service walks the Trie, sorts results, writes them to Redis, and returns them.
- Trie vs linear search: a linear scan over N terms takes $O(N * L)$ where L is word length; a Trie lookup is much faster for large vocabularies because the prefix narrows the search space quickly.

LEARNING OUTCOMES

- Implemented an autocomplete service using Spring Boot.
- Built a Trie for prefix-based search.
- Returned top-K suggestions sorted by frequency.
- Integrated Redis as a cache for frequently requested prefixes.
- Understood the performance benefit of caching over repeated Trie searches.